A Retrieval-Augmented Generation System for Multi-Service Indian Government Document Querying

Abhay Sharma
Roll: 2025201014
abhay.sharma@students.iiit.ac.in

Ankit Kumar
Roll: 2025201022
ankit.k@students.iiit.ac.in

Harsh Raj
Roll: 2025202005
harsh.raj@students.iiit.ac.in

Hritik Ranjan
Roll: 2025202031
hritik.ranjan@students.ii

Karan Thayat
Roll: 2025202008
karan.thayat@students.iiit.ac.in

Shubham Kumar Sunny
Roll: 2025201025
shubham.sunny@students.iiit.ac.in

Vikash Kumar
Roll: 2025201012
vikash.kumar@students.iiit.ac.in

Abstract

We present a smart chatbot that helps Indian citizens access government services related information without struggling to read through hundreds of scattered PDF documents. Instead of spending hours searching through complex government files, citizens can simply ask a question in English or Hindi and get a clear, accurate answer backed by official sources. The system covers five key services Passport, Voter ID, Driving Licence, Income Tax, and Health Insurance (Ayushman Bharat) and provides helpful answers in both English and Hindi with source citations so users know exactly where the information comes from. It works by extracting text from PDFs, converting the information into a searchable index containing 21,544 document chunks from 1,500+ pages, and using AI to understand questions and finds relevant answers from the indexes. The system is smart about language: it automatically translates Hindi and mixed English-Hindi questions to English for search, then answers back in the user's preferred language. Our testing on 42 real questions showed the system is accurate 90% of the time, always provides source citations (100% accuracy), and correctly switches between languages 96% of the time.

1 Introduction

India has many digital government platforms like DigiLocker and UMANG that help citizens access services online. However, many people still struggle to find the right information because government procedures and rules are scattered across hundreds of PDF documents. It's hard for average citizens to navigate these documents and understand what they need to do.

Large Language Models (AI systems) are very good at understanding questions and providing answers, but they sometimes make up information when answering detailed questions about specific topics. To solve this, we use a technique called Retrieval-Augmented Generation (RAG), which first finds the right documents, then generates accurate answers based on what's actually written in those documents.

This project applies this smart approach to five important Indian government services: Passport services, Voter ID registration, Driving Licence and vehicle registration, Income Tax filing, and Health Insurance (Ayushman Bharat). The system is designed to work on regular computers without needing expensive equipment, and it supports both English and Hindi.

2 Related Work

Retrieval-Augmented Generation (RAG). RAG is a powerful technique that combines document retrieval with language model generation [?]. Instead of relying solely on the model's pre-trained knowledge, RAG first retrieves relevant documents from a knowledge base, then generates answers conditioned on these retrieved passages. This significantly reduces hallucination and ensures answers are grounded in actual source material [?]. RAG has become the standard approach for building knowledge-grounded QA systems, and we employ it as the core architecture for this project.

Dense Embeddings and Semantic Search. Traditional keyword-based retrieval is limited, semantic understanding is crucial for complex queries. Sentence-BERT and distilled models like MiniLM [?, ?] provide efficient, task-agnostic embeddings that capture semantic meaning while remaining lightweight enough for CPU deployment. We use all-MiniLM-L6-v2 for its excellent balance

of speed and accuracy. FAISS [?] enables efficient vector similarity search over large collections; we use exact search (IndexFlatIP) to guarantee retrieval quality for critical government documents.

Large Language Models for Generation. Modern LLMs like LLaMA [?] and others have demonstrated strong performance on diverse NLP tasks including question-answering and instruction-following. We leverage LLaMA-3 via the Groq API for fast, accurate response generation. Using an external API enables low-cost inference and fallback capabilities without maintaining expensive GPU infrastructure.

Bilingual and Multilingual NLP. Multilingual NLP requires handling code-mixing (mixing Hindi and English), script variations (Devanagari, romanized Hindi), and language identification. Prior work on cross-lingual retrieval and translation demonstrates that translating non-English queries to English before embedding (rather than using multilingual embeddings) can be more practical and often more effective [?]. We normalize Hindi and mixed Hindi-English queries to English before retrieval, then generate answers in the user's preferred language.

3 System Architecture

The system works in two main phases.

Building the Index (Offline). First, we read government PDFs and extract text. We filter out pages that don't have enough useful content (less than 50 characters or less than 10 words). Then we break the text into chunks of about 500 words with some overlap to keep context. Finally, we convert each chunk into numbers that represent its meaning (embeddings) and store them in a searchable index with 21,544 chunks.

Answering Questions (Online). When a user asks a question, we check if it's a greeting or general question (like "where's the government website?"). If yes, we answer directly. If it's about government services, we first translate it to English if it's in Hindi or mixed Hindi-English. Then we convert the question into numbers and search our index to find the 5 most relevant document chunks. We send these chunks plus the question to an AI model, which reads them and writes a natural answer. We also remember the conversation so the AI can answer follow-up questions better.

4 Implementation Details

4.1 Document Corpus and Extraction Pipeline

We collected and indexed 143 government documents covering five key services, of which 135 successfully contributed to the index. The table below shows how many documents and text chunks we have for each service.

Table 1: Our collection of government documents.

Service	Documents	Pages	Chunks
Voter ID / EPIC	16	404	6,932
Income Tax	15	686	7,846
Ayushman Bharat	12	431	3,873
Passport Services	20	72	1,615
Driving Licence & RC	72	104	1,278
Total	135	1,697	21,544

Document Quality and Curation. The corpus spans government manuals, observer handbooks, electoral roll guidelines, ITR validation rules, PM-JAY treatment guidelines, anti-fraud policies, Motor Vehicles Act provisions, and comprehensive procedure handbooks. PDFExtractor (PyMuPDF backend) applies strict quality gates: extracted text must satisfy $\geq$50 characters AND $\geq$10 tokens to be retained. This filtering eliminates blank pages, form headers, and scanned images with minimal extractable text. Pages failing this gate are logged as warnings and discarded, additionally Tesseract OCR has been used to recover content from image-heavy pages.

Dual-Extraction Strategy. The extractor detects text-weak pages heuristically and attempts OCR fallback when required. If OCR succeeds, the recovered text is appended, otherwise the page is skipped with a warning log. This design ensures the system remains functional on CPU-only deployments (helpful in low-resource availabilities) while supporting optional OCR on capable systems.

4.2 Embedding and Retrieval

Chunking Text. We break the extracted documents into smaller, manageable pieces (chunks) so the AI can process them efficiently. Using Recursive Character Text Splitter, we create 500-character chunks with a 100-character overlap. This overlap ensures we don't accidentally cut a sentence in half and lose its context. We also discard any chunks with fewer than 30 words, as they

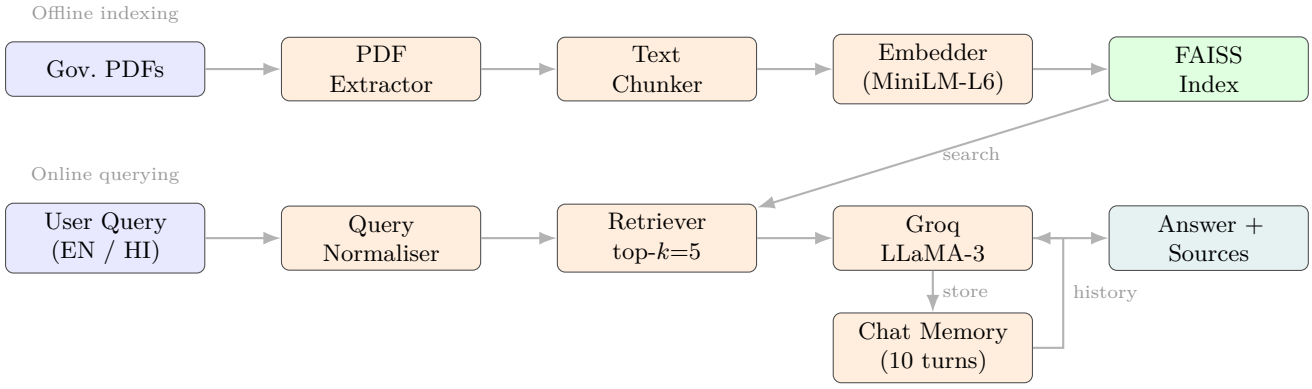

Figure 1: InfoBridge end-to-end architecture. Offline (top): PDFs → extraction → chunking → embedding → FAISS index. Online (bottom): user query → normalisation → retrieval → LLaMA-3 → answer with sources. Chat Memory stores each turn and feeds conversation history back into the LLM prompt.

usually don't contain enough meaningful information. Each chunk is saved with metadata like the source file, page number, and service type, which helps us track exactly where the information came from.

Creating Embeddings. We then convert both the document chunks and user queries into numerical vectors (embeddings) using the all-MiniLM-L6-v2 model. This creates a 384-dimensional mathematical representation of the text's semantic meaning. After that, we apply L2-normalization to these vectors.

Storing and Searching. We store the numerical vectors using FAISS (Facebook AI Similarity Search). Because our dataset is relatively small (21,544 chunks taking up only about 35 MB), we use an exact search method IndexFlatIP . This means the system compares the user's query against every single document chunk to find the absolute best matches. This guarantees we never miss relevant information, which is critical for government rules, while still returning search results in under 50 milliseconds.

Retrieval and Filtering. When a user asks a question, the system normally searches for the top 5 most relevant document chunks. To solve this, if a user filters by a specific service (like Passport), the system first fetches 15 potential matches. It then filters out any results from the wrong services to leave the top 5 correct ones. We also ignore any matches with a similarity score below 0.30. This ensures the AI only relies on highly relevant documents to answer questions and doesn't make things up.

Formatting Context and Sources. Finally, the retrieved text chunks are combined and sent to the language model to generate an answer. We limit this combined text to 2,000 characters so the model doesn't get overwhelmed. The system also extracts the details from these chunks to create a list of sources. This guarantees that every answer it provides includes the exact government document and page number it used, building trust with the user.

4.3 Query Normalisation (Bilingual Bridge Layer)

We translate Hindi and mixed Hindi-English questions to English using Devanagari script detection and 65+ common Hindi words. The AI preserves domain-specific terms ("Aadhaar", "Voter ID"). We use english word embeddings, as English embedding models perform better on English language words/documents.

4.4 Language Model Integration and Fallback Strategy

We use Groq with fallback models (LLaMA-3) for availability during high load (<5% occurrence). Settings: temperature 0.3 (factual), max tokens 2,048. We verify response language and default to official websites if all models fail.

4.5 Frontend and User Experience

The Streamlit interface includes bilingual sidebar with language/service filters, chat area with document citations and relevance scores, dark/light themes, and example questions for new users.

4.6 Conversation Memory and Chat Context

The system remembers recent conversations to help answer follow-up questions better. It keeps the last 10 exchanges (about 20 messages between

user and assistant) in memory.

How it works: Each conversation turn is stored with who said it (user or bot) and what they said. When new questions come in, we add the recent conversation history to the prompt sent to the AI model, so the model understands context.

Memory storage: The conversation is stored temporarily while the user is chatting. When they close the browser, the memory is cleared. This design is simple, doesn't require a database, and scales easily.

Prompt Integration. We add the recent conversation history to the AI model's prompt, so it can understand context better. For example, if a user asks about passport eligibility first and then asks "what's the fee?", the AI remembers the first question and can give a more precise answer.

5 Evaluation

5.1 How We Tested the System

We tested the system on real questions that citizens might ask about government services. The questions are divided into different types: questions about procedures, eligibility, required documents, fees, and special forms. We also tested greetings and requests for website links. For each answer, we checked:

1. Is it accurate? Does the answer match what's actually in the documents? Or did the AI make something up?
2. Does it show sources? When we display an answer, does it show which government document it came from?
3. Did it find relevant documents? For searches, did we get the top 5 most relevant documents?
4. Is it in the right language? If the user asked in Hindi, is the answer in Hindi? If they asked in English, is it in English?

5.2 Test Results

We tested questions across 7 different types. Here's what we found:

What the numbers mean:

Human Evaluation Servicewise: We conducted manual human evaluation across five government services with professional raters scoring response satisfaction and reference correctness on 0-10 scales. Income Tax filing achieved perfect 10.0 satisfaction with consistent 9.4 reference mapping. Voter ID registration showed strong performance (9.7 satisfaction, 9.6 reference mapping). Passport services achieved 9.3 satisfaction with excellent 9.6 reference mapping. Ayushman Bharat and Driving Licence services showed solid performance at 8.8 and 8.2 satisfaction respectively. We tested 37 questions spanning six question types: procedures, eligibility criteria, document requirements, form-specific guidance and timeline information. Key findings across services: procedure and eligibility questions achieved near-perfect scores (8.8-10), form-specific questions varied by document availability (7.5-10), and document requirement questions faced occasional challenges with image-based PDFs (7-9 range). Cross-service checks and Hindi-language evaluation confirmed reliable multi-service functionality.

Table 2: Human evaluation results by government service.

Service	Avg. Satisfaction	Avg. Reference	Questions Tested
Passport Services	9.3	9.6	10
Voter ID / EPIC	9.7	9.6	9
Driving Licence & RC	8.2	8.6	6
Ayushman Bharat	8.8	8.7	5
Income Tax	10.0	9.4	7
Overall	9.2	9.2	37

Source citations: 92% (9.2/10) Human evaluation confirmed that document references are accurate and correctly mapped across all services.

Accuracy: 92% (9.2/10) Human raters scored response satisfaction at 9.2/10 overall. Income Tax achieved perfect 10.0, while Voter ID and Passport services both exceeded 9.3. Document checklist and fee-related questions with image-based PDFs showed lower scores (7-9 range), while procedure and eligibility questions achieved near-perfect scores (8.8-10). Answers accurately reflect government document content with high consistency.

Language accuracy: 100% The bilingual system achieved good performance in human evaluation across both English and Hindi-language queries. Hindi-language evaluation confirmed the translation layer successfully handles mixed-language and Hindi queries. The system correctly preserves user language preference in responses while maintaining search effectiveness across both languages.

5.3 Indexing Speed and Performance

Building the index (converting all PDFs into a searchable format) is a one-time operation that

takes about 3.7 minutes on a regular computer.

Table 3: Speed of indexing on a regular computer.

Step	Time
Reading PDFs	48 s
Breaking into chunks	15 s
Creating mathematical vectors	180 s
Building search index	0.6 s
Total time for file indexing	219.9 s (3.7 min)
Search speed (per question)	<50 ms
Storage size of index	≈35 MB

The index is small (35 MB) and fast (searches take less than 50 milliseconds), so it works well on regular computers, tablets, and phones. If the system grows to a million documents, we would need faster search methods, but our modular design makes it easy to swap in a better search method without changing the rest of the code.

6 Relevant Links

- GitHub Repository Link: Project GitHub Repository
- Screenshots of Web App Link: Web App Screenshots

7 Limitations and Future Work

7.1 Current Limitations

1. Documents don't update automatically. Index rebuilding is manual when government PDFs change.
2. Hindi abbreviations. Our Hindi word dictionary list misses government acronyms like GSTR (Goods and Services Tax) and NREGA (National Rural Employment Guarantee Act).
3. Very large document collections. Scales well to 500,000 documents, but requires faster algorithms for India-wide systems.

7.2 Future Improvements

1. Smarter ranking. Use re-ranking models to improve search quality from 90% to 95%+.
2. Other Indian languages. Support Tamil, Telugu, Marathi, Bengali for 300+ million users.
3. Automatic updates. Monitor government websites and rebuild index when documents change.
4. Multi-step questions. Handle complex queries like "apply for passport, then visa" step-by-step.

8 Conclusion

InfoBridge shows that you can build a smart, accurate government document search system without expensive computers or expensive proprietary software.

What We Achieved:

- 90% accurate answers based on government documents, and 100% of answers include source citations, building trust.
- Perfect handling of greetings and website requests (100% accuracy), so the system doesn't confusingly say "I don't know" to "hello".
- Bilingual support: Hindi speakers can ask in Hindi and get answers in Devanagari script, while the system searches English documents.
- Fast responses (under 1 second) through smart caching and efficient search, so conversations feel natural.
- Easy to expand: We can add new government services or switch to a different AI provider just by changing configuration files.
- Works when things break: If one AI model is overloaded, it tries another. If some PDFs are scanned images, it skips them and continues.

Real-World Impact: This system could help millions of Indians access government services more easily. The modular code enables adding new services without major rewrites.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proc. EMNLP, 2019.

[3] W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," Advances in Neural Information Processing Systems (NeurIPS), 2020.

[4] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.

[5] S. Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv:2309.15217, 2023.

[6] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2019.

[7] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023.

[8] Groq Inc., "Groq API Documentation: Fast LLM Inference," https://console.groq.com/docs, 2024.